Prueba técnica · Desarrollador Python Junior

Área de Analítica · Soluciones de movilidad urbana

Antes de empezar: cómo está pensada esta prueba

Esperamos que uses inteligencia artificial. Nos interesa incorporar a quienes usan estas herramientas de forma activa y saben sacarles provecho. Usa la que prefieras, todo el tiempo que quieras.

Precisamente por eso, **no calificamos solo que el código funcione**. Hoy cualquiera genera código que parece funcionar. Calificamos que **entiendas lo que entregas**: por qué está escrito así, qué casos cubre, qué casos no, y qué harías si mañana cambia una regla.

Después de la entrega hay una sesión de 45 minutos por videollamada. En ella:

- Recorres tu código y explicas tus decisiones.
- Hacemos un **cambio pequeño en vivo** sobre tu repositorio. Puedes usar IA, pero nos vas contando qué haces y por qué.
- Hay un segmento corto **sin IA**: te mostramos fragmentos cortos de Python y nos dices qué hacen. No es de memoria ni de trucos raros; es de entender el lenguaje que usas todos los días.

Tiempo: dispones de 48 horas calendario. El esfuerzo esperado es de **4 a 6 horas de trabajo real**. Si lo resuelves bien en menos, mejor.

El contexto

La **Línea T2** es un corredor de transporte público de 14 estaciones. El servicio `tarifas-t2` calcula cuánto se le cobra a una tarjeta por sus validaciones (cada vez que el usuario pasa la tarjeta por un torniquete).

Hoy el servicio cobra **tarifa completa en cada validación**. La ciudad aprobó la **tarifa integrada con transbordos** y hay que implementarla. Además, soporte reportó un bug que hay que corregir.

El repositorio que recibes es el código actual, con su historial de git. Trátalo como código heredado de un compañero: puede tener cosas buenas y cosas que no lo son.

Encargo 1 · Implementar la tarifa con transbordos

Reglas de negocio

1 · Viaje. Una validación que no es transbordo inicia un viaje nuevo y paga la **tarifa base: $3.200**.

2 · Transbordo. Una validación que ocurre dentro de los **90 minutos siguientes a la primera validación del viaje** (límite incluido) es un transbordo.

3 · Valor del transbordo. El primer transbordo de un viaje paga **$500**. El segundo es **gratis**. No hay tercer transbordo: a partir de ahí, la validación inicia un viaje nuevo y paga tarifa base.

4 · Perfiles. Cada tarjeta tiene un perfil. El descuento se aplica a **cada cobro** (tarifa y transbordo):

Perfil	Descuento
`general`	0%
`estudiante`	35%
`adulto_mayor`	55%

5 · Redondeo. Cada cobro, después del descuento, se redondea al **múltiplo de $50 más cercano**. Si queda exactamente en la mitad, se redondea **hacia arriba**. Se redondea cada cobro por separado, no el total.

6 · Las validaciones llegan como las mandan los validadores. Los validadores de los buses sincronizan cuando tienen señal. En `datos_ejemplo/` tienes una muestra real de cómo llegan.

Contrato del endpoint

El endpoint ya existe; debes mantener este contrato exacto, porque otro sistema lo consume.

`POST /tarifas/calcular`

```
{
  "tarjeta": "T-004417",
  "perfil": "general",
  "validaciones": [
    {"id": "a1f0", "estacion": "E01", "fecha_hora": "2025-03-10T06:52:10-05:00"},
    {"id": "a1f1", "estacion": "E07", "fecha_hora": "2025-03-10T07:31:44-05:00"}
  ]
}
```

Respuesta `200`:

```
{
  "tarjeta": "T-004417",
  "perfil": "general",
  "cobros": [
    {"validacion_id": "a1f0", "fecha_hora": "2025-03-10T06:52:10-05:00",
     "tipo": "tarifa", "valor": 3200},
    {"validacion_id": "a1f1", "fecha_hora": "2025-03-10T07:31:44-05:00",
     "tipo": "transbordo", "valor": 500}
  ],
  "total": 3700
}
```

- `cobros` va en **orden cronológico**.
- `tipo` es uno de: `tarifa`, `transbordo`, `transbordo_gratis`.
- `valor` y `total` son **números enteros** en pesos.
- `perfil` es opcional; si no viene, es `general`.
- Si no hay validaciones, la respuesta es `200` con `cobros` vacío y `total` 0.

Responder `422` si:

- el perfil no es uno de los tres válidos;
- alguna `fecha_hora` **no trae zona horaria** (no hay forma de saber a qué hora ocurrió);
- hay dos validaciones con el **mismo** `id`.

Encargo 2 · Corregir un bug reportado

> ***BUG-17 · Prioridad alta*** *"Un usuario llamó a reclamar: al consultar el historial de su tarjeta (*`GET /tarjetas/{tarjeta}/historial`*) le aparecen cálculos de otras tarjetas. Pasa en producción. En local, con una sola tarjeta, no lo pudimos reproducir."*

Corrígelo. Queremos que nos expliques **por qué pasaba**, no solo qué cambiaste.

Encargo 3 · Que corra en Docker

El repositorio trae `Dockerfile` y `docker-compose.yml`. Con

```
docker compose up --build
```

el servicio debe quedar respondiendo en `http://localhost:8000/docs` desde tu navegador. Verifícalo de verdad, no asumas que funciona.

Qué debes entregar

Un único `.zip` con tres cosas:

A. El repositorio, con su historial de git

- Incluye la carpeta `.git`. Vamos a leer tu historial.
- Trabaja en una rama (por ejemplo `feature/transbordos`) y haz commits que cuenten la historia: uno por cambio lógico, con mensajes que digan qué y por qué. Un solo commit gigante con "cambios" nos dice poco.
- **Tests:** agrega los tests que consideres necesarios para demostrar que las reglas se cumplen. `pytest` debe pasar sin errores. El proyecto trata los *warnings* como errores; no lo desactives.
- No incluyas `.venv`, `__pycache__` ni archivos de tu IDE.

B. `NOTAS.md` en la raíz del repo — máximo 1 página

- Qué encontraste en el código heredado que te llamó la atención, además del bug reportado.
- Los casos borde que identificaste y cómo los resolviste.
- Cualquier supuesto que hayas tenido que hacer.
- Qué dejarías para después si esto fuera a producción.

C. Bitácora de uso de IA — máximo 1 página · obligatoria

Sin este documento la entrega no se evalúa. Debe incluir:

1. Qué herramienta o herramientas usaste.
2. Los tres prompts que consideres mejores, transcritos.
3. **Si en algún momento no estuviste de acuerdo con lo que la IA propuso, o algo no te cuadró: qué fue, cómo lo notaste y qué hiciste.** Si no se presentó el caso, cuéntanos cómo verificaste lo que te entregó.
4. Qué aportaste tú que la IA no habría aportado sola.

Canal de preguntas

Durante las 48 horas puedes escribirnos con cualquier duda, respondiendo al correo con el que recibiste esta prueba. Respondemos en menos de 4 horas hábiles. **Preguntar no resta puntos. Preguntar bien suma.**

Cómo se evalúa

Dimensión	Peso
Correctitud — las reglas se cumplen, incluidos los casos borde	25
Conceptos de Python — segmento sin IA en la sesión	20
Defensa y cambio en vivo — explicas tu código y lo modificas	20
Tests — qué probaste y por qué	10
Git y Docker	10
Bitácora de IA	10
`NOTAS.md`	5

Fíjate que la mitad del puntaje sale de la sesión. Un código perfecto que no puedes explicar no aprueba.

Una última cosa

Si en algún momento piensas "esto parece raro, pero seguro está bien", detente y revísalo. Esa sensación suele ser la parte importante del problema.

Éxitos.